

Agentic AI System for Enterprise Cloud Migration

Timothy Wong • Alessandro Abati

London Stock Exchange Group (LSEG)

7 Oct 2026

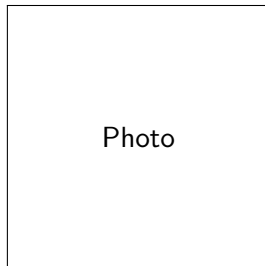


About the Speakers



Timothy Wong

Tim is an AI/ML engineer at LSEG with over ten years experience in enterprise data science applications. He hosted talks at EARL 2016, 17, 19 & 24.



Alessandro Abati

placeholder



LSEG

LSEG is a leading financial markets infrastructure and data provider operating connected businesses to serve customers across the entire financial markets value chain.

- ▶ **London Stock Exchange**
- ▶ **FTSE Russell:** Indices across asset classes, including FTSE 100, FTSE 250, Russell 2000, and ESG indices.
- ▶ **Data & Analytics:** Workspace platform, pricing, reference data.
- ▶ **Clearing:** LCH, multi-asset class clearing house.
- ▶ **Scale:** ~26 000 employees across many countries.

1. **Business Requirements and Context**
2. **Agentic Technical Deep Dive**
 - 2.1 **Data Preprocessing**
 - 2.2 **Inside Agentic Retrieval**
 - 2.3 **Agentic Orchestration**
 - 2.4 **Agents' Memory**
 - 2.5 **Agents' Tools**
 - 2.6 **E2E Orchestration Phases**

- ▶ **Problem:** *placeholder* — what makes enterprise cloud migration slow, risky and manual.
- ▶ **Goal:** *placeholder* — automate discovery, analysis and recommendation with AI agents.
- ▶ **Stakeholders & scope:** *placeholder*.
- ▶ **Success criteria:** *placeholder*.

- ▶ **Ingestion:** *placeholder* — raw sources into Azure Blob / Table storage.
- ▶ **Knowledge sources & bases:** *placeholder* — Azure AI Search skillsets.
- ▶ **Chunking / enrichment:** *placeholder*.
- ▶ **Output:** *placeholder* — indexed data ready for agentic retrieval.

- ▶ **What it is:** *placeholder* — Azure AI Search agentic retrieval vs. classic RAG.
- ▶ **Query planning:** *placeholder*.
- ▶ **Retrieval flow:** *placeholder*.
- ▶ **Grounding:** *placeholder*.

Agentic Orchestration — Overview

- ▶ **Framework:** *placeholder* — Microsoft Agent Framework.
- ▶ **Orchestration patterns used:** GroupChatManager, Concurrent, Writer/Reviewer.
- ▶ **When each pattern applies:** *placeholder*.

- ▶ **Pattern:** *placeholder* — manager coordinates a conversation between agents.
- ▶ **Turn / termination control:** *placeholder*.
- ▶ **Where used:** *placeholder*.

- ▶ **Pattern:** *placeholder* — fan-out agents run in parallel batches.
- ▶ **Aggregation:** *placeholder*.
- ▶ **Where used:** deep analysis batches, category-scoped UAQ agents.

- ▶ **Pattern:** *placeholder* — writer drafts, reviewer critiques, loop to convergence.
- ▶ **Quality gates:** *placeholder*.
- ▶ **Where used:** reporting phase.

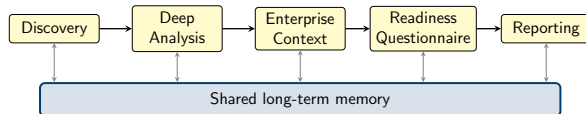
Agents' Memory

Shared long-term memory

- ▶ A single shared memory accumulates findings across the whole run.
- ▶ Each phase reads what earlier phases discovered and writes back its own results.
- ▶ Later phases build on earlier work instead of re-reading the raw inputs.

Passing context between phases

- ▶ The pipeline is serial: every phase produces a report.
- ▶ Each report, with the shared memory, becomes the input context for the next phase.
- ▶ By the final phase, agents reason over the full accumulated picture.



Reports flow forward; every phase reads from and writes to the shared memory.

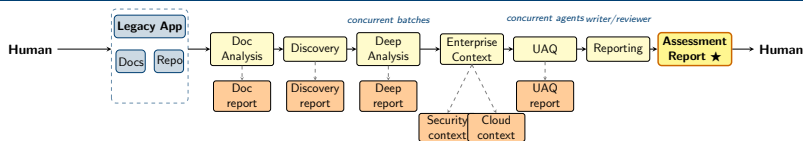
What tools are available

- ▶ **File access:** browse and read the codebase and documents.
- ▶ **Knowledge retrieval:** ask questions over indexed documents; answers cite their sources.
- ▶ **Enterprise knowledge:** search organisational guidance (security, cloud, DevOps standards).
- ▶ **Diagrams:** create, validate and fix architecture diagrams, rendered offline.

How agents use them

- ▶ Each agent is given only the tools relevant to its task.
- ▶ Tools turn raw inputs into structured, grounded findings.
- ▶ Grounded retrieval keeps answers anchored to real evidence.
- ▶ Tool use is logged for observability and cost tracking.

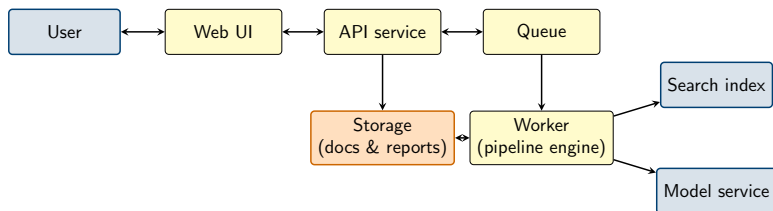
E2E Orchestration Phases



Inputs (documents & code) enter on the left; each phase records its findings and passes context forward.

- ▶ **Discovery:** group the code and documents into logical components. *(Output: a map of the system.)*
- ▶ **Deep Analysis:** assess each component's migration readiness, in parallel. *(Output: per-component findings.)*
- ▶ **Enterprise Context:** apply organisational security and cloud standards. *(Output: policy-aligned recommendations.)*
- ▶ **Readiness Questionnaire:** answer a standard cloud-readiness questionnaire. *(Output: structured answers.)*
- ▶ **Reporting:** consolidate and review all findings. *(Output: the final assessment report.)*

Infrastructure & Backend



- ▶ **Flow:** users upload through the web UI; the API stores inputs and queues a job; a worker then runs the phase pipeline.
- ▶ **Shared services:** the worker draws on storage, the search index and the model service to complete each phase.
- ▶ **Isolated environment:** the system runs fully offline, with automatic retries on transient failures.

Thank you — Questions?